GAMING CENTER MANAGEMENT

System Documentation

Prepared by:

Aldrin Antony

Msccs2505
Rollno:05

Technology: Java SE | Swing | Oracle DB

1. Project Title

Gaming Center Management System

A desktop-based Java Swing application connected to an Oracle Database for managing the complete operations of a gaming center — customer registration, game catalogue, session bookings, and payment collection.

2. Introduction

Gaming centers require an efficient system to track customers, manage their game preferences, handle session bookings, and process payments. Manual record-keeping is error-prone and slow. The Gaming Center Management System addresses these challenges by providing a centralized, professional desktop application that automates these workflows.

2.1 Objectives

- Manage customer records (add, update, delete, view).
- Maintain a catalogue of available games with pricing information.
- Allow staff to create session bookings linking customers to games.
- Record and track payments for completed bookings.
- Provide a professional, intuitive Neon Arena themed user interface.

2.2 Technology Stack

Component	Technology
Frontend (UI)	Java Swing (JFrame, JTable, JPanel, etc.)
Backend Logic	Java SE 25
Database	Oracle Database (JDBC via ojdbc11.jar)
IDE	Apache NetBeans
Build System	Apache Ant (build.xml)

2.3 Scope

The system covers four core modules: Customer Management, Game Management, Booking Management, and Payment Processing. All operations are persisted immediately to the Oracle Database using JDBC.

3. Class Diagram

The diagram below illustrates the object-oriented design: model classes, DAO (Data Access Object) classes, their attributes, methods, and relationships.

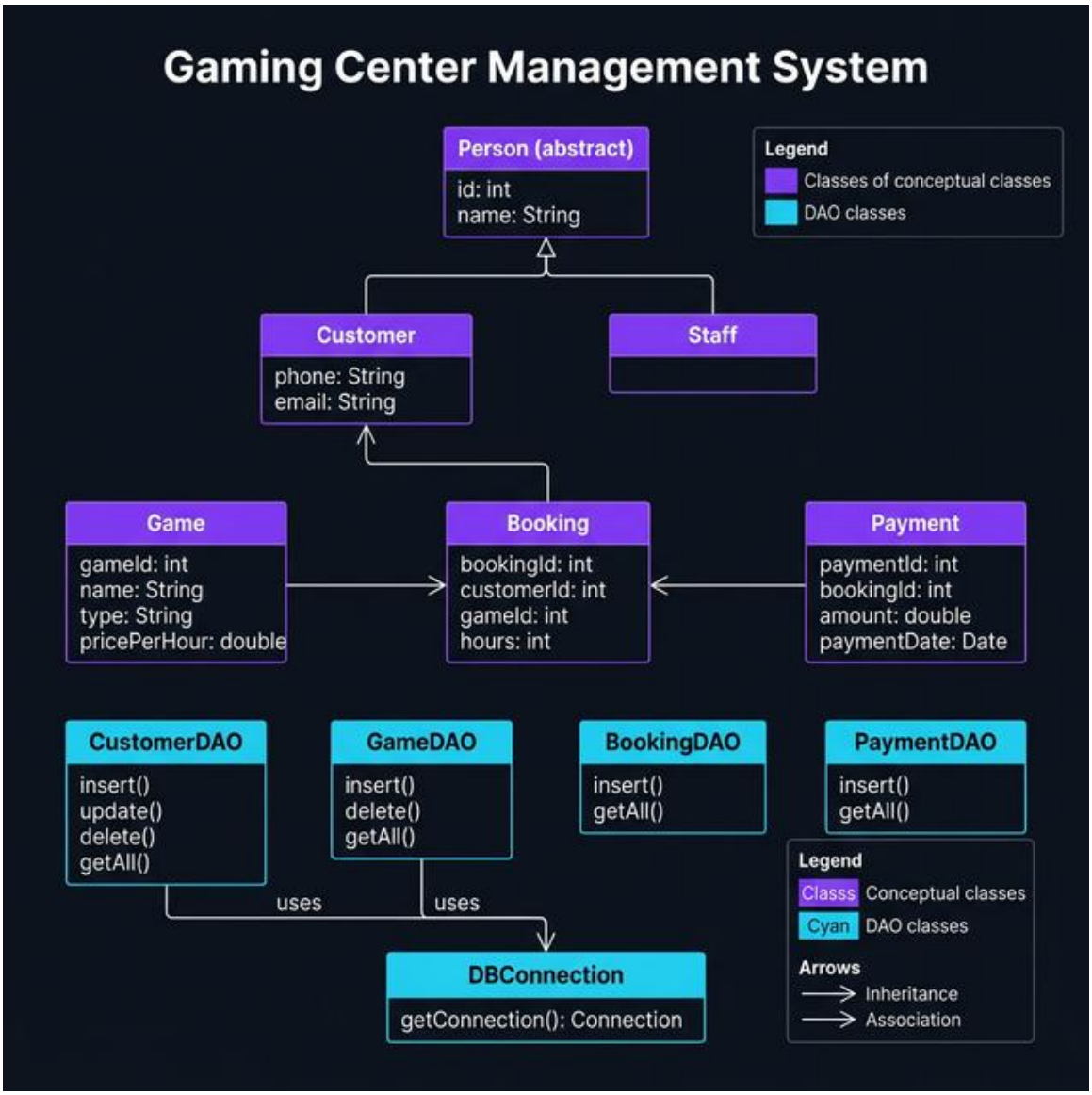


Figure 1 – UML Class Diagram (Gaming Center Management System)

3.1 Class Descriptions

Person (Abstract): Base class containing shared attributes id and name. Extended by Customer.

Customer: Stores customer contact details: phone and email. Inherits from Person. Persisted via CustomerDAO.

Game: Represents a game in the catalogue with name, type, and pricePerHour. Managed via GameDAO.

Booking: Links a Customer to a Game for a specified number of hours. References customerId and gameId as foreign keys.

Payment: Records a financial transaction for a Booking. Stores amount and payment_date (auto-set to Oracle SYSDATE).

DBConnection: Singleton utility that provides a JDBC Connection to Oracle DB.

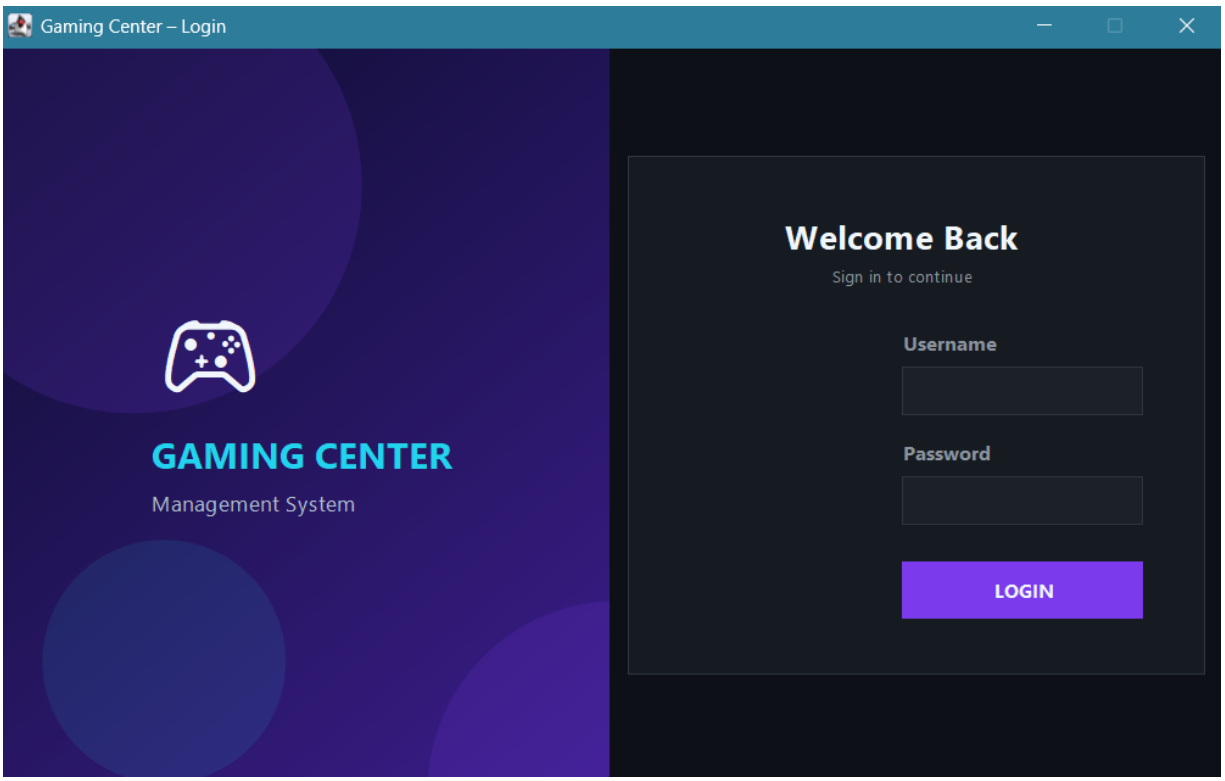
CustomerDAO / GameDAO / BookingDAO / PaymentDAO: Data Access Objects encapsulating all SQL logic, keeping business logic separate from the UI layer.

4. Application Screenshots

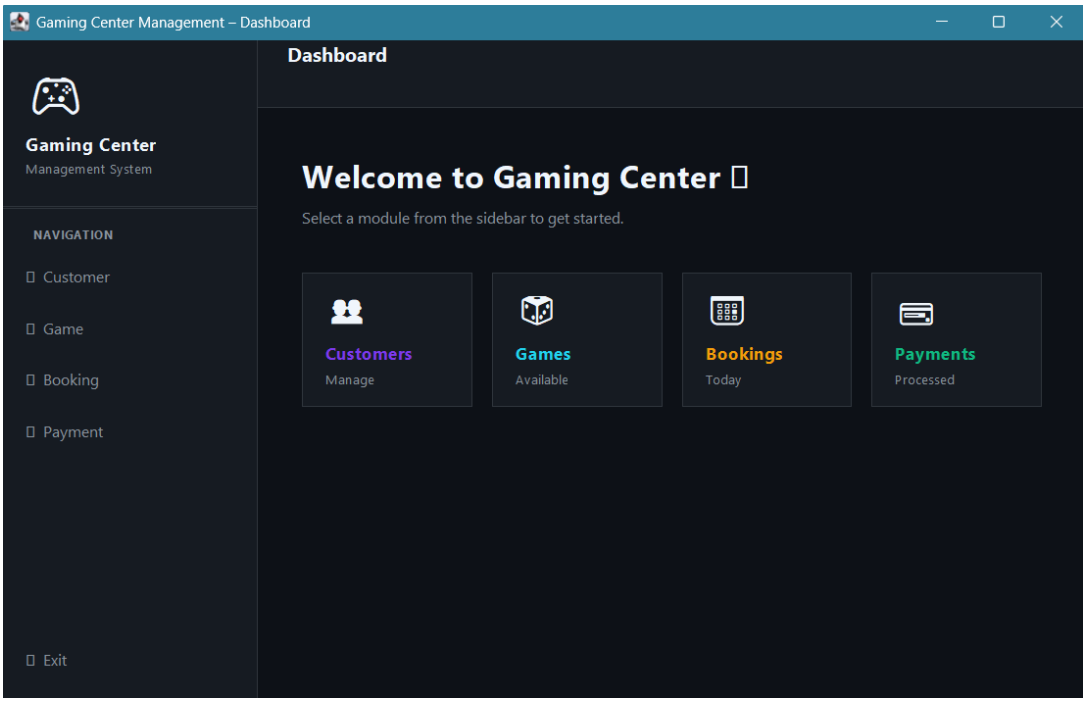
The following placeholders represent each screen of the application. Replace each grey box with the actual screenshot taken from the running app.

4.1 Login Screen

Gradient branded panel on the left and a dark login card on the right with username/password fields and a LOGIN button.



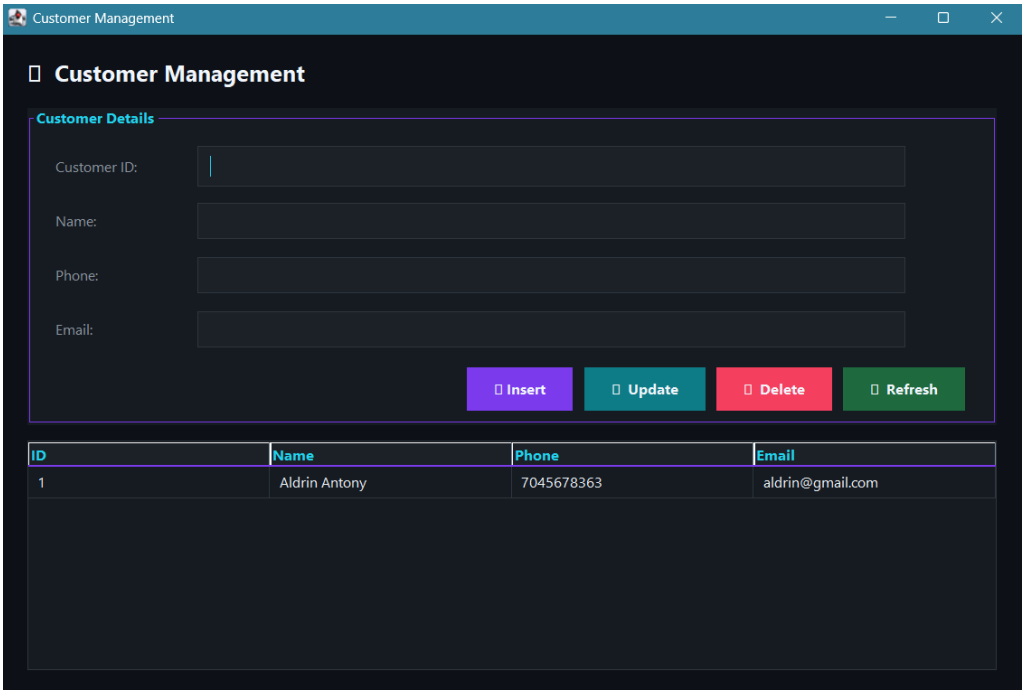
4.2 Dashboard – Main Menu



Left sidebar with emoji navigation links (Customer, Game, Booking, Payment, Exit) and a welcome area with four stat cards.

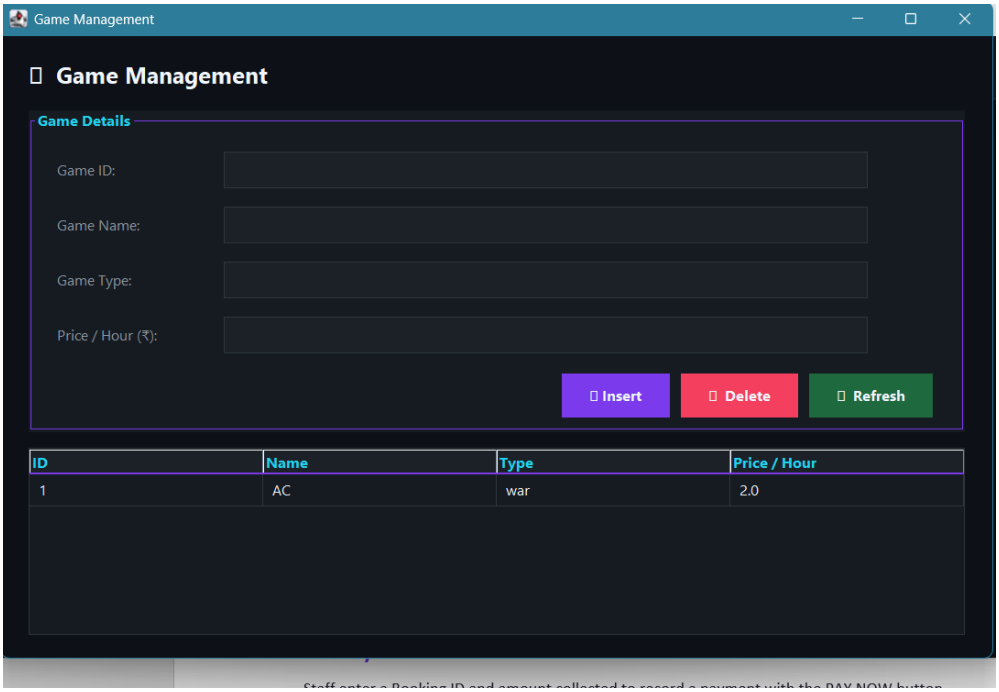
4.3 Customer Management

Form card for Add/Update/Delete/Refresh with a live data table below. Clicking a table row auto-fills the form fields.



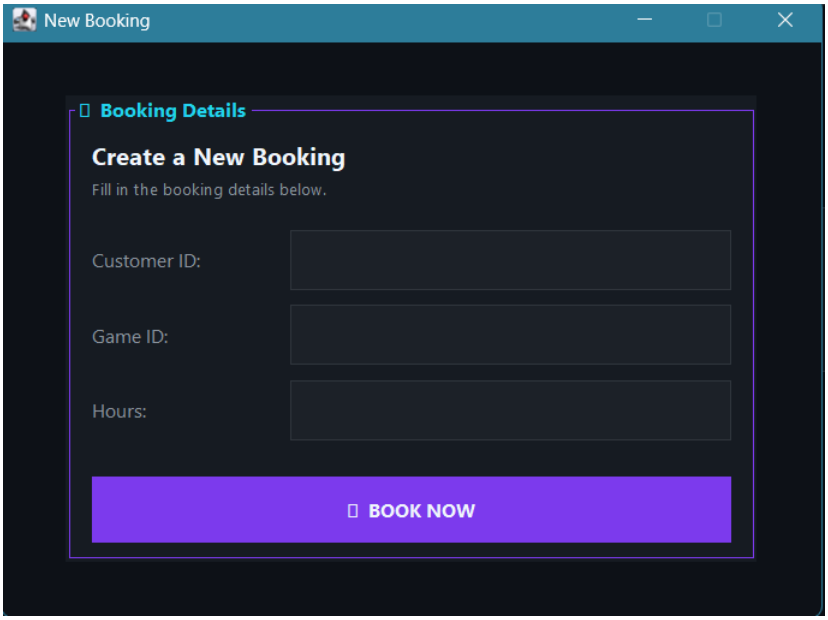
4.4 Game Management

Form for adding and deleting games (name, type, price per hour) with a live table that loads on open.



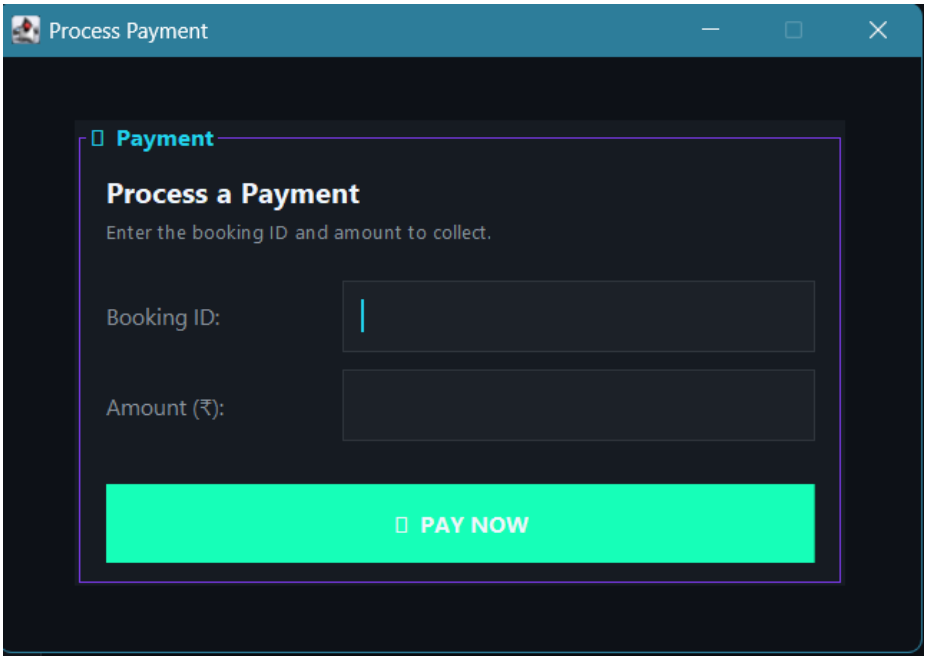
4.5 Booking

Staff enter Customer ID, Game ID, and number of hours to create a session booking via the BOOK NOW button.



4.6 Payment

Staff enter a Booking ID and amount collected to record a payment with the PAY NOW button.



5. Database Design

The system uses Oracle Database. Primary keys are generated via Oracle sequences (e.g. customers_seq.NEXTVAL). Foreign key relationships enforce referential integrity between tables.

USERS

Table: USERS
username VARCHAR2(50)
password VARCHAR2(50)

CUSTOMERS

Table: CUSTOMERS
customer_id NUMBER (PK – customers_seq)
name VARCHAR2(100)
phone VARCHAR2(20)
email VARCHAR2(100)

GAMES

Table: GAMES

game_id NUMBER (PK – games_seq)
name VARCHAR2(100)
type VARCHAR2(50)
price_per_hour NUMBER

BOOKINGS

Table: BOOKINGS
booking_id NUMBER (PK – bookings_seq)
customer_id NUMBER (FK → customers)
game_id NUMBER (FK → games)
hours NUMBER

PAYMENTS

Table: PAYMENTS
payment_id NUMBER (PK – payments_seq)
booking_id NUMBER (FK → bookings)
amount NUMBER
payment_date DATE